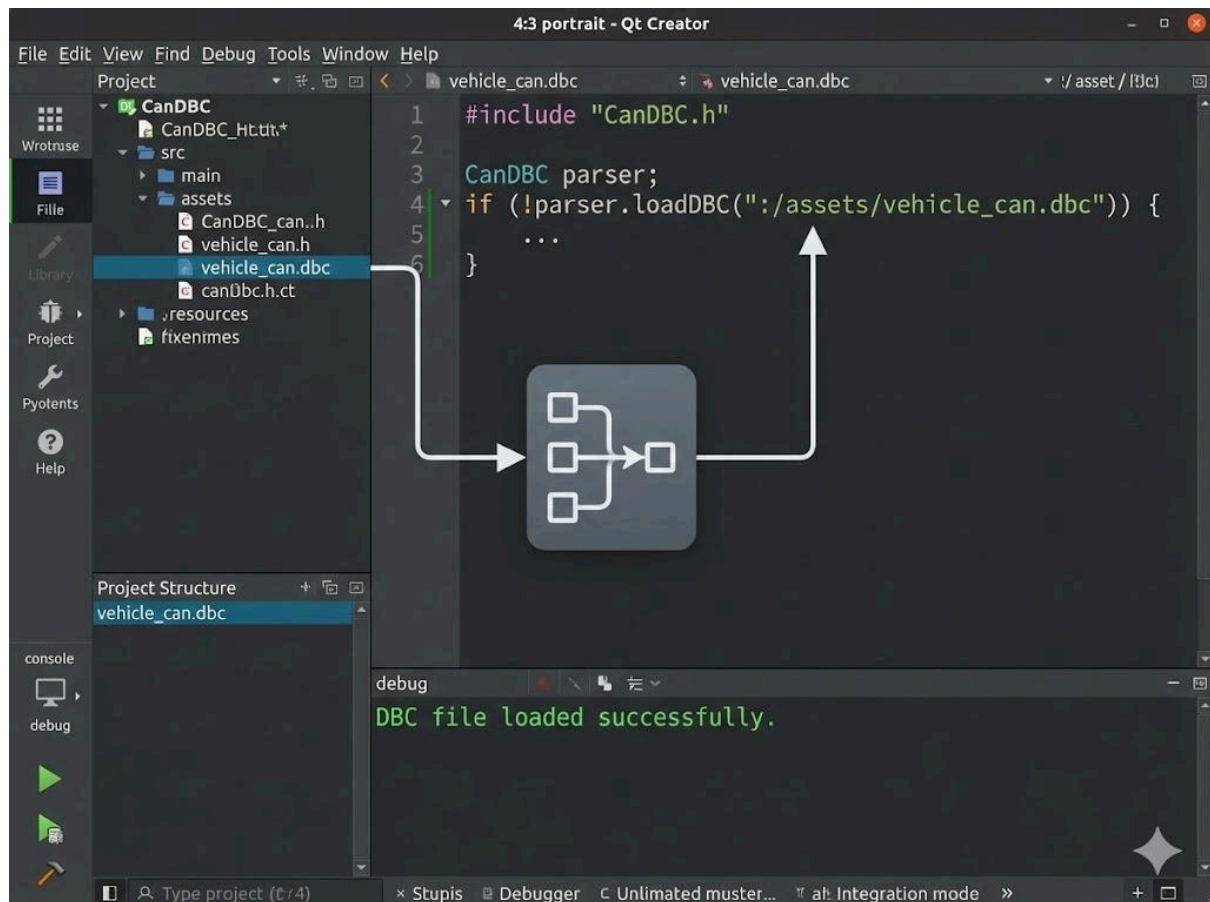


1. Initialization

This image visualizes the first step of loading the DBC file. It shows the Qt Creator IDE with the initialization code, the `.dbc` file in the project structure, and a successful load message in the application output.



2. Inspecting the Database

This image demonstrates how to inspect the loaded DBC data. It displays the code for iterating through the message list, the resulting debug output with message names and IDs, and a watch window showing the structured data.

The screenshot displays a Qt IDE interface with three main components:

- Code Editor:** Shows a C++ code snippet for iterating through a list of messages.
- Application Output:** Displays the debug output from the code, showing message names and IDs.
- Watch Window:** Shows the structured data of the 'messages' variable, including a list of items with their names and IDs.

```
35  
36 const QList<CanDBC::Message>& messages = parser.msgList();  
37 for(const auto& msg : messages)  
38 {  
39     qDebug() << ... message() << msgList;  
40 }  
41
```

Application Output

messages (2) x

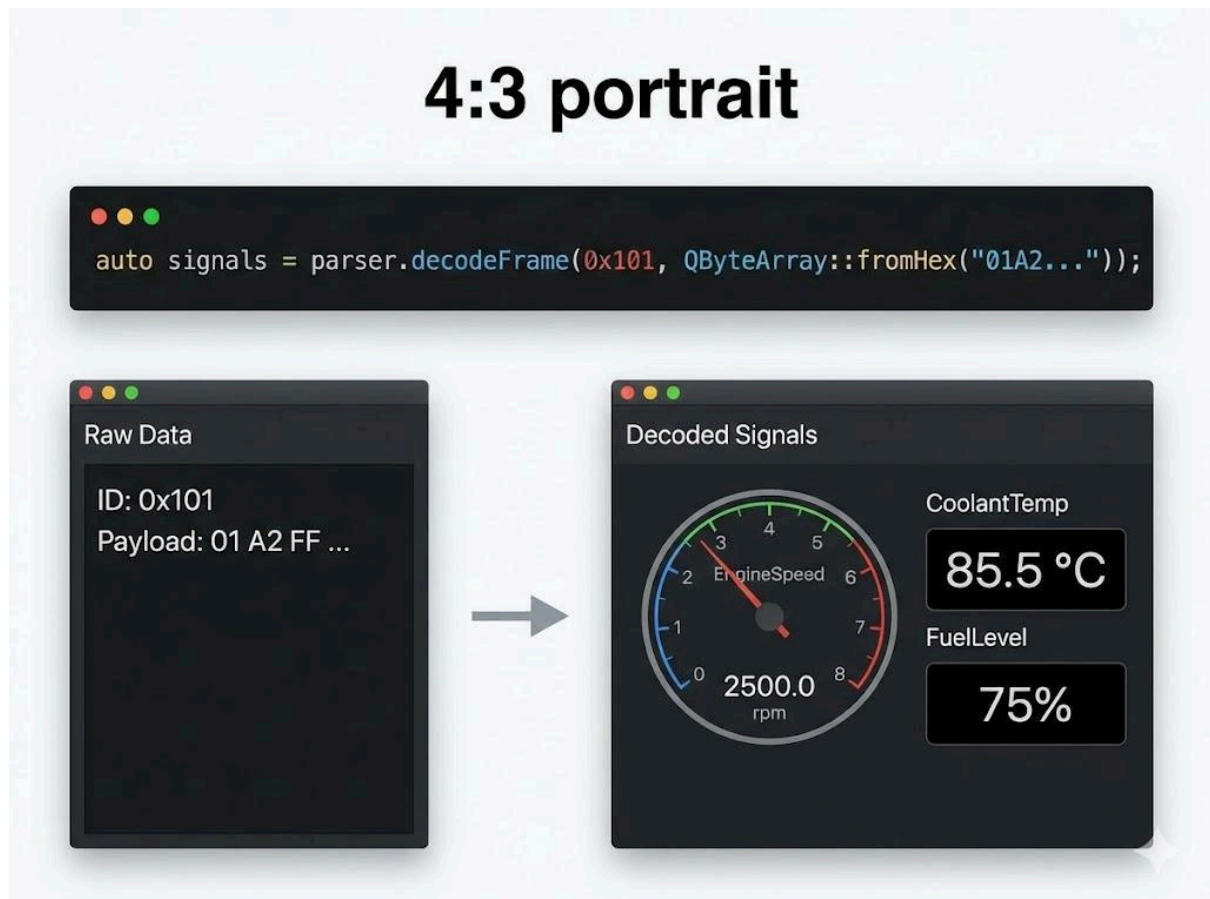
Message: EngineData ID: 0x101
Message: EngineData ID: 0x101
Message: EngineData ID: 0x101
Message: EngineData ID: 0x101
Message: TransmissionStatus ID: 0x205
Message: TransmissionStatus ID: 0x205
Message: ABS_Status ID: 0x310
Message: ABS_Status ID: 0x310
Message: ABS_Status ID: 0x310
Message: ABS_Status ID: 0x320
Message: EngineData ID: 0x101
Message: TransmissionStatus ID: 0x205
Message: ABS_Status ID: 0x310

Watch

Name	Value
oneration	CanDBC::Me...
variable	[null]
messages	CanDBC::Me...
object	Object CanD...
{0} itemName	EngineData
{1} item:ID	EngineData
{2} itemName	Transmission...
{3} itemName	EngineData
{4} itemName	Transmission...
{5} itemName	EngineData...
{6} walr:ID	0x101
{7} healthyValue	Object CanD...
{8} itemName	ABS_Status
{9} item:ID	Transmission...
{0} itemName	ABS_Status
{1} itemName	0x110
{0} walr:ID	0
{1} healthyValue	Object
{2} itemName	EngineDataD...
{3} item:ID	Object CanD...
{4} itemName	Object Conte...
{5} itemName	0x310

3. Real-time Decoding

The final image illustrates the real-time decoding process. It shows the code for decoding a raw CAN frame and visualizes the transformation of raw hex data into human-readable physical signals on a dashboard-style interface.



4. Connect & Follow

This image encourages users to star the repo and connect with you for updates. It features the LinkedIn and GitHub logos with a clear "Follow for More" visual cue.

